



Figure 1: Memory segments of a C program

```

    printf("[test3] global_string = %s\n", global_string);
}
int main(int argc, char **argv){
    test1();
    test2();
    test3();
    return 0;
}

```

Let's compile the program with `gcc test.c -o test` and use `'nm'` to dissect it:

```

$ nm ./test
00049674 d _DYNAMIC
00049740 d _GLOBAL_OFFSET_TABLE_
0004855c R _IO_stdin_used
          w _Jv_RegisterClasses
00049664 d __CTOR_END__
00049660 d __CTOR_LIST__
0004966c D __DTOR_END__
00049668 d __DTOR_LIST__
0004865c r __FRAME_END__
00049670 d __JCR_END__
00049670 d __JCR_LIST__
00049764 A __bss_start
0004975c D __data_start
00048510 t __do_global_ctors_aux
00048370 t __do_global_dtors_aux
00048500 R __dso_handle
          w __gmon_start__
0004850a T __i686.get_pc_thunk.bx
00049660 d __init_array_end
00049660 d __init_array_start
000484a0 T __libc_csu_fini
000484b0 T __libc_csu_init
          U __libc_start_main@@GLIBC_2.0
00049764 A __edata
0004977c A __end
0004853c T __fini

```

```

00048558 R _fp_hw
000482b4 T _init
00048340 T _start
00049764 b completed.5963
00048564 R const_int
0004975c W data_start
00049768 b dtor_idx.5965
000483d0 t frame_dummy
00049778 B global_int1
00049760 D global_int2
0004976c B global_string
00048476 T main
          U memcpy@@GLIBC_2.0
          U printf@@GLIBC_2.0
000483f4 T test1
0004841d T test2
00048459 T test3

```

Let's examine the output. First, where do the program functions go? Near the end of the output, you see the names `main`, `test1`, `test2` and `test3` with a 'T' preceding them; 'T' stands for the text section, in which all these functions are. Next, the global and static variables: `global_string` and `global_int1` are preceded with 'B', while we have 'R `const_int`' and 'D `global_int2`'. This is because the data section is divided into two further sections: Uninitialised Data or BSS (Block Start by Symbol), and Initialised Data. Both '`global_string`' and '`global_int1`' are declared but not initialised, so they are in BSS ('B'); '`global_int2`' has been initialised, and is in 'D', the Initialised Data section.

The storage of '`const_int`' is interesting. We declared it as a `const` variable, whose value won't change throughout the program—a read-only variable. Thus, it is stored in 'R', the read-only data section.

There are options for `nm` (see the man page, and experiment, to understand object files). For example, `'-S'` will show the size of each symbol in hexadecimal form, as follows (a snippet of the output):

```

$ nm -S ./test
00049778 00000004 B global_int1
0004976c 0000000a B global_string
00048476 0000001e T main

```

Objdump

The next utility I will cover is '`objdump`'. As a kernel developer, most of the time I have to deal with kernel crashes. Debugging the kernel needs a good understanding of the relation between C and assembly. For learning, working on a kernel crash would be time-consuming, so let's start with a user-space program. Let's compile `test.c` with debugging